

Functional Requirements Specification (FRS)

System: Jima — CARAVAN Lounge Restaurant Management System
Document: FRS v1.2 · **Updated:** August 20, 2026
Related documents: SRS (02), SDD (03), User Manual (04), Handoff (05), deployment guide (06), and operational runbooks (07–12)

1. Purpose

This document defines the functional requirements of the Jima Restaurant Management System — a full-stack web application that manages the daily operations of CARAVAN Lounge: ordering, kitchen workflow, payments, inventory, staff, branches, and reporting.

Verified implementation baseline: the registered Jima web artifact runs the real Next.js 15 / React 19 frontend in `artifacts/nextjs-app`. Browser requests use `/backend/api/*`, which Next.js rewrites to the NestJS service's `/api/*`. NestJS and TypeORM own all PostgreSQL access. The separate `artifacts/api-server` is not the restaurant application API.

Requirements versus evidence: the items below define required behavior. A requirement must not be treated as production-verified unless it appears as passed evidence in document 12.

2. User Roles

Role	Description
Admin	Full system access; manages all data, users, and configuration.

Owner	Business owner; same operational visibility as manager, including analytics dashboard.
Manager	Runs daily operations; manages menu, staff, tables, branches, inventory, and reports.
Coordinator	Confirms incoming orders and assigns them to chefs; can cancel orders.
Waiter	Takes orders at tables (POS), adds items to open orders, tracks order progress.
Chef	Works the kitchen board: accepts, prepares, and marks orders ready.
Cashier	Collects payments, closes orders, views sales and items-purchased reports.
Storekeeper	Manages stock levels, receives goods, handles item requests and adjustments.

3. Functional Requirements

3.1 Authentication & Authorization

ID	Requirement
FR-A01	Users log in with e-mail and password; the system returns a JWT access token.
FR-A02	Every protected API call requires a valid <code>Authorization: Bearer</code> token. Login and health endpoints are intentional public exceptions; administrative seed operations must remain protected.
FR-A03	Menu entries and pages are restricted per role (role-based access control).
FR-A04	Passwords are stored hashed (bcrypt), never in plain text.

3.2 Orders & POS

ID	Requirement
----	-------------

FR-O01	Waiters create orders through a POS screen: pick a table (or take-away), select menu items by category, set quantities and notes.
FR-O02	Orders follow the lifecycle: pending → confirmed → preparing → ready → served → paid , with cancelled possible before "ready".
FR-O03	The coordinator confirms a pending order and assigns a chef to it.
FR-O04	Waiters may append items to any order that is not yet paid or cancelled.
FR-O05	The orders list shows table, chef, item count, time, total, payment method, and status; rows open a detail view.
FR-O06	Orders can be filtered by status, waiter, table, item, date, time, and payment method.
FR-O07	The list and open detail view refresh automatically every 10 seconds.
FR-O08	On phone-sized screens, the POS keeps menu and checkout controls usable by stacking the menu above the cart, preserving horizontal category browsing, and providing separate vertical scrolling for menu items and checkout content.

3.3 Kitchen

ID	Requirement
FR-K01	The kitchen board shows confirmed orders queued per chef, with tabs per chef.
FR-K02	Chefs move orders through accept → preparing → ready.
FR-K03	The Order Progress Board displays four live columns: Requested, Received & Confirmed, In Progress, Completed (ready only).
FR-K04	Orders waiting more than 20 minutes are visually flagged.

3.4 Payments

ID	Requirement
FR-P01	

	Cashiers record payments against served orders with method cash, card, or wallet, including change given.
FR-P02	Paying an order marks it paid and frees its table.
FR-P03	Daily reports summarize revenue, transaction counts, and per-method breakdowns.

3.5 Menu Management

ID	Requirement
FR-M01	Managers maintain menu categories and items (name, price, description, availability, preparation time).
FR-M02	Unavailable items are hidden from the POS.

3.6 Inventory & Store

ID	Requirement
FR-I01	Inventory items track unit, current stock, minimum stock, reorder point, unit cost, category, and expiry date.
FR-I02	Low-stock alerts flag items below minimum (critical when at or below half the minimum, or zero).
FR-I03	Suppliers and purchase orders are managed with status tracking.
FR-I04	Stock adjustments record additions, deductions, and waste with reasons and the acting user.
FR-I05	Staff submit item requests; managers/storekeepers approve, issue, and track them, with low-stock / expiry badges shown on request rows.

3.7 Staff, Branches & Tables

ID	Requirement
FR-S01	Managers create and manage staff accounts with roles and branch assignment.

FR-S02	Restaurants and branches are managed hierarchically (restaurant → branches).
FR-S03	Tables have number, capacity, section, and live status (available / occupied).

3.8 Dashboards & Reports

ID	Requirement
FR-D01	Admin/Owner/Manager see a combined dashboard: today's KPIs (sales, orders, average order value, in-progress, low stock, stock value), sales trend, top sellers, profitability, and kitchen performance — on a single page. KPI cards open detail popups.
FR-D02	Reports offer per-area export tabs (Order Board, Menu, Inventory, Item Requested, Staff, Branches) with filters and Excel / PDF export.
FR-D03	Cashiers see a reduced report view: Sales and Items Purchased only, filterable by type and date, exportable to Excel / PDF.
FR-D04	Role-specific landing: waiters, coordinators, and cashiers land on Orders/POS instead of the dashboard.

3.9 Notifications

ID	Requirement
FR-N01	The system generates notifications targeted at roles or specific users (e.g. new order, order ready) with read tracking.

4. Business Rules

- An order's total is the sum of its items (quantity × unit price) plus any service charge percentage.
- Only pending orders can be confirmed; only served orders can be paid.
- Cancelling is allowed only while an order is pending, confirmed, or preparing.

- Data is scoped by branch where applicable (orders, tables, staff, stock).

5. Verification Status

Functional development is active and major workflows have been exercised manually. Complete automated regression evidence for every role, branch boundary, order transition, payment, inventory operation, notification, report, and export is not yet recorded.

Production release remains blocked until the verification matrix in document 12 is complete.

Software Requirements Specification (SRS)

System: Jima — CARAVAN Lounge Restaurant Management System
Document: SRS v1.2 · **Updated:** August 20, 2026
Standard: Structured after IEEE 830

1. Introduction

1.1 Purpose

This SRS specifies the complete requirements — functional and non-functional — for the Jima Restaurant Management System serving CARAVAN Lounge. It is intended for developers, testers, and stakeholders taking over or extending the system.

1.2 Scope

The system digitizes end-to-end restaurant operations: table-side ordering (POS), kitchen workflow, cashiering, inventory and procurement, staff and branch administration, and management analytics. It is a web application accessed from browsers on desktops, tablets, and phones.

1.3 Definitions

Term	Meaning
POS	Point of Sale — the screen waiters use to compose orders.
Order Board	Live kanban-style view of order progress.
JWT	JSON Web Token used for stateless authentication.

Item Request	An internal request for stock items from the store.
--------------	---

2. Overall Description

2.1 Product Perspective

Three-tier logical architecture: a **Next.js 15 / React 19 frontend**, a **NestJS REST API backend**, and a **PostgreSQL database** accessed through TypeORM. The registered Jima web artifact runs the actual source in `artifacts/nextjs-app`. The frontend calls the backend through `/backend/api/*`; Next.js rewrites requests to the backend's `/api/*`. The separate `artifacts/api-server` is not the restaurant API.

2.2 User Classes

Eight roles: admin, owner, manager, coordinator, waiter, chef, cashier, storekeeper (see FRS §2). Users are non-technical restaurant staff; the UI must remain simple, large-touch-friendly, and in plain language.

2.3 Operating Environment

Layer	Technology
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS, Zustand (auth store), Recharts, Axios
Backend	NestJS 10, TypeScript, Passport-JWT, bcrypt, TypeORM
Database	PostgreSQL 14+ (TypeORM <code>synchronize: true</code> in development)
Runtime	Node.js 18+, pnpm monorepo workspaces

2.4 Constraints

- Protected API access requires JWT. Login and health are public; seed operations are administrative.
- The current backend still enables TypeORM schema synchronization. Production is blocked until synchronization is disabled there and a tested migration/rollback baseline exists.

- Realtime behavior is implemented by 10-second polling, not WebSockets.
- Currency is Ethiopian Birr (ETB); demo data uses Ethiopian dishes and names.

3. Functional Requirements

The complete functional catalogue is maintained in the FRS (document 01). Summary of subsystems:

Subsystem	Capabilities
Auth	Login, JWT issuance, role-based menus and guards
Orders / POS	Create, append items, confirm + assign chef, status lifecycle, filters, auto-refresh
Kitchen	Chef queues, accept/preparing/ready, live progress board with delay flags
Payments	Cash/card/wallet payment capture, change, daily report, shifts
Menu	Categories and items with availability
Inventory	Stock, low-stock alerts, suppliers, purchase orders, adjustments, item requests
Admin	Staff, restaurants, branches, tables
Analytics	Manager/owner one-page dashboard, exportable reports (Excel/PDF), cashier-scoped reports
Notifications	Role/user-targeted messages with read tracking

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	

		Standard pages should render within 2 s on a typical broadband connection; list endpoints return within 500 ms for up to 10,000 orders.
NFR-02	Freshness	Operational views (orders, kitchen, boards, dashboards) auto-refresh every 10 seconds without flicker.
NFR-03	Security	JWT-based auth; bcrypt password hashing; role guards on every protected endpoint; secrets provided via environment variables, never committed.
NFR-04	Usability	Plain-language UI, color-coded statuses, and touch-friendly controls. The Orders/POS flow must remain usable from a 320 px phone viewport upward: no overlapping checkout controls, intentional horizontal scrolling only for category tabs or data tables, and reachable checkout actions through vertical scrolling.
NFR-05	Reliability	Backend failures surface explicit errors; the frontend degrades gracefully (empty states, retry on next poll).
NFR-06	Maintainability	TypeScript end-to-end; modular NestJS feature modules; monorepo with shared conventions; <code>tsc --noEmit</code> must pass.
NFR-07	Portability	Runs anywhere Node 18+ and PostgreSQL are available; deployable to Replit, Vercel (frontend) + managed Node host (backend), or a single VM.
NFR-08	Auditability	Stock adjustments and item requests record acting users and timestamps.

5. External Interface Requirements

5.1 User Interface

Single-page-app feel with a persistent left sidebar (role-filtered), light gray theme with brand-orange accents (#E8832A), and modal dialogs

for POS, payment, order detail, and confirmations. On phones, the Orders/POS workspace stacks the menu above the cart and keeps checkout controls scrollable and reachable.

5.2 API Interface

REST over HTTPS with JSON bodies. The browser-facing path is `/backend/api`; the backend's direct prefix is `/api`. Interactive Swagger documentation exists at backend path `/api/docs` and must be restricted or disabled in production. Main resource groups are `/auth`, `/users`, `/restaurants`, `/branches`, `/menus`, `/tables`, `/orders`, `/kitchen`, `/payments`, `/inventory`, `/notifications`, `/summary`, `/seed`.

5.3 Database Interface

PostgreSQL via TypeORM repositories; connection string in `DATABASE_URL`; SSL honored when `sslmode=require` is present.

6. Assumptions & Dependencies

- A single restaurant company operates the system; multiple branches are supported.
- Kitchen staff record stock deductions/waste for profitability estimates to be accurate.
- Exports (Excel/PDF) are generated client-side (xlsx, jsPDF) and require a modern browser.

7. Acceptance and Production Evidence

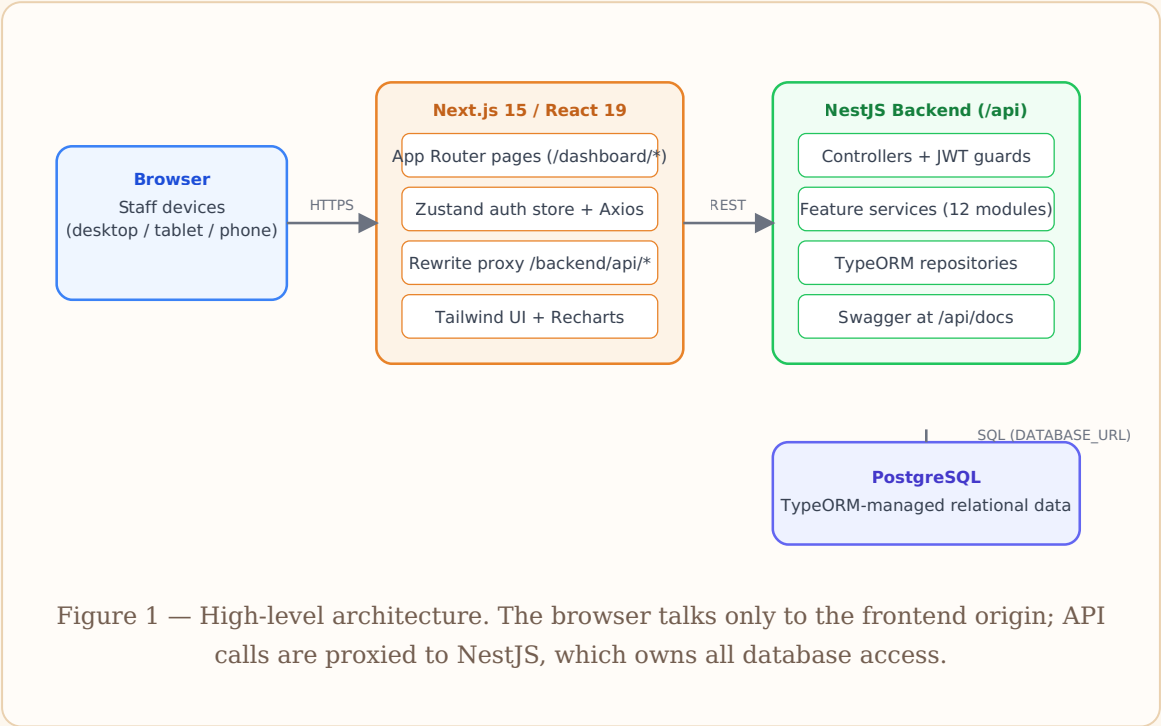
Architecture assessment is complete, but repository evidence for the full safe-refactoring, separation, automated-regression, security-hardening, migration, backup/restore, and production-verification stages is incomplete. Requirements marked in this SRS are acceptance targets, not automatic claims of production readiness. Refer to document 12 for current READY / NEEDS ATTENTION / BLOCKED status.

Software Design Document (SDD)

System: Jima — CARAVAN Lounge Restaurant Management System
Document: SDD v1.2 · **Updated:** August 20, 2026

1. System Architecture

The system is a three-tier logical application and modular monolith. The registered Jima web artifact runs `artifacts/nextjs-app` (frontend). `artifacts/nestjs-server` is the authoritative restaurant backend. `artifacts/api-server` is a separate minimal artifact and must not be confused with the restaurant API.



2. Frontend Structure

Path	Responsibility
------	----------------

<code>src/app/login</code>	Login page (photo hero, credential form).
<code>src/app/dashboard/page.tsx</code>	Role-aware landing: renders ManagerDashboard for admin/owner/manager; redirects waiter/coordinator/cashier to Orders.
<code>src/app/dashboard/*</code>	Feature pages: orders (POS), order-board, kitchen, tables, menu, inventory, item-requests, requests, staff, branches, reports. The order page provides a stacked phone layout for its POS workspace.
<code>src/components</code>	Shared UI: sidebar (role-filtered nav), manager-dashboard, stat cards, etc.
<code>src/lib/api.ts</code>	Browser API facade; attaches JWT and calls the same-origin <code>/backend/api</code> boundary.
<code>src/store/auth.ts</code>	Zustand store: current user, token, login/logout.

Data freshness is achieved with a uniform pattern: each operational page runs `fetchData()` on mount and every 10 s via `setInterval`, without toggling loading state (no flicker).

3. Backend Structure

Module	Endpoints (prefix <code>/api</code>)	Notes
Auth	POST <code>/auth/login</code> , GET <code>/auth/me</code>	bcrypt verify → JWT (Passport strategy)
Users	<code>/users</code> , <code>/users/staff-list</code> , <code>/users/waiters</code> , <code>/users/chefs</code>	Role management
Restaurants / Branches	<code>/restaurants</code> , <code>/branches</code>	Company hierarchy
Menus	<code>/menus</code> (categories + items)	Availability flags feed POS
Tables	<code>/tables</code>	Status transitions
Orders	<code>/orders</code> (+ <code>/stats</code> , <code>/alerts</code> , item edits, status)	Lifecycle enforcement
Kitchen		Chef queues

	/kitchen (board, accept/preparing/ready)	
Payments	/payments (+ report, shifts)	Marks orders paid, frees tables
Inventory	/inventory (items, low-stock, suppliers, purchase orders, requests, adjustments)	Store operations
Notifications	/notifications	Role/user targeted
Seed	POST /seed	Demo data (Ethiopian menu, staff)

4. Database Design (ER Overview)

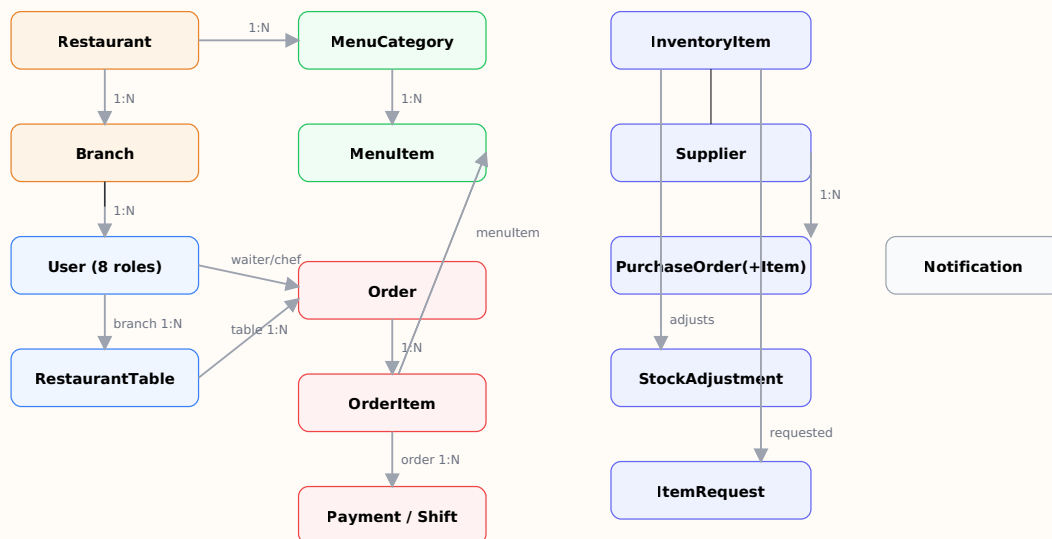


Figure 2 — Entity-relationship overview (key relations only). All entities carry auto-increment ids and timestamps where relevant.

Key columns per entity are listed in the SRS §2.3 environment and in the backend entity files (`artifacts/nestjs-server/src/**/*entity.ts`), the authoritative source. Notable choices:

- `Order.status` enum drives the entire workflow; `OrderItem` cascades on order delete.
- `User.password` uses `select:false` so hashes never leak into queries by default.

- `ItemRequest` keeps requester / approver / issuer user references for auditability.

5. Order Lifecycle (Sequence)

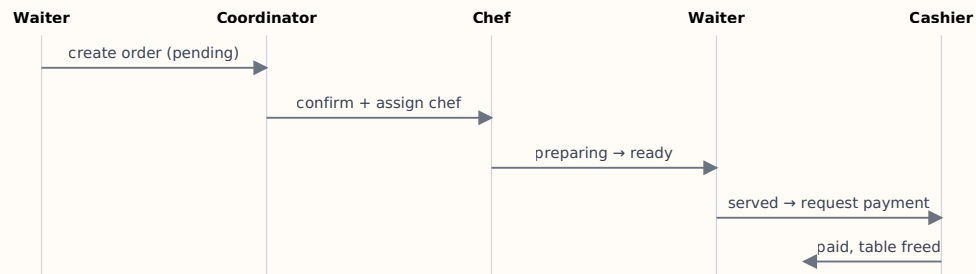


Figure 3 — Happy-path order sequence across roles.

6. Cross-Cutting Design Decisions

Concern	Decision
Auth	Stateless JWT (Passport). Token in the Zustand store; Axios interceptor adds the header.
API routing	Backend global prefix <code>/api</code> ; frontend proxies <code>/backend/api/*</code> via Next.js rewrites so the browser stays same-origin.
Realtime	10-second polling everywhere (simple, proxy-friendly, no WebSocket infra).
Schema	TypeORM <code>synchronize:true</code> for development speed; switch to migrations for production (see Handoff doc).
Exports	Client-side generation: <code>xlsx</code> for Excel, <code>jspdf</code> + <code>jspdf-autotable</code> for PDF, loaded dynamically.
Styling	Tailwind CSS with shared utility classes (<code>.card</code> , <code>.input</code> , <code>.table-cell</code>) in <code>globals.css</code> ; brand color <code>#E8832A</code> .
Mobile POS layout	At phone widths the POS shell changes from a side-by-side menu/cart layout to a vertical layout. Menu items and the bounded cart/check-out area scroll independently; cart fields use a single-column layout and long item names wrap instead of forcing horizontal

	overflow. The desktop two-pane layout remains in place at large widths.
Profitability	Food cost estimated from stock deduction/waste movements × item unit cost (documented in User Manual).

7. Boundary and Readiness Notes

- The frontend owns presentation, navigation, forms, loading/error states, and API communication. It must never receive database credentials or own authoritative business permissions.
- The NestJS backend owns authentication, authorization, role/branch scope, validation, transactions, business rules, and all database access.
- PostgreSQL owns persistence and relational integrity. TypeORM entities are currently authoritative, but production migration evidence is not yet complete.
- The browser contract remains `/backend/api/*`; changing it requires coordinated frontend, rewrite, backend, deployment, and documentation updates.
- Current production blockers include schema synchronization, a fallback JWT secret, startup account mutation, wildcard CORS, incomplete DTO validation, and incomplete automated tests.

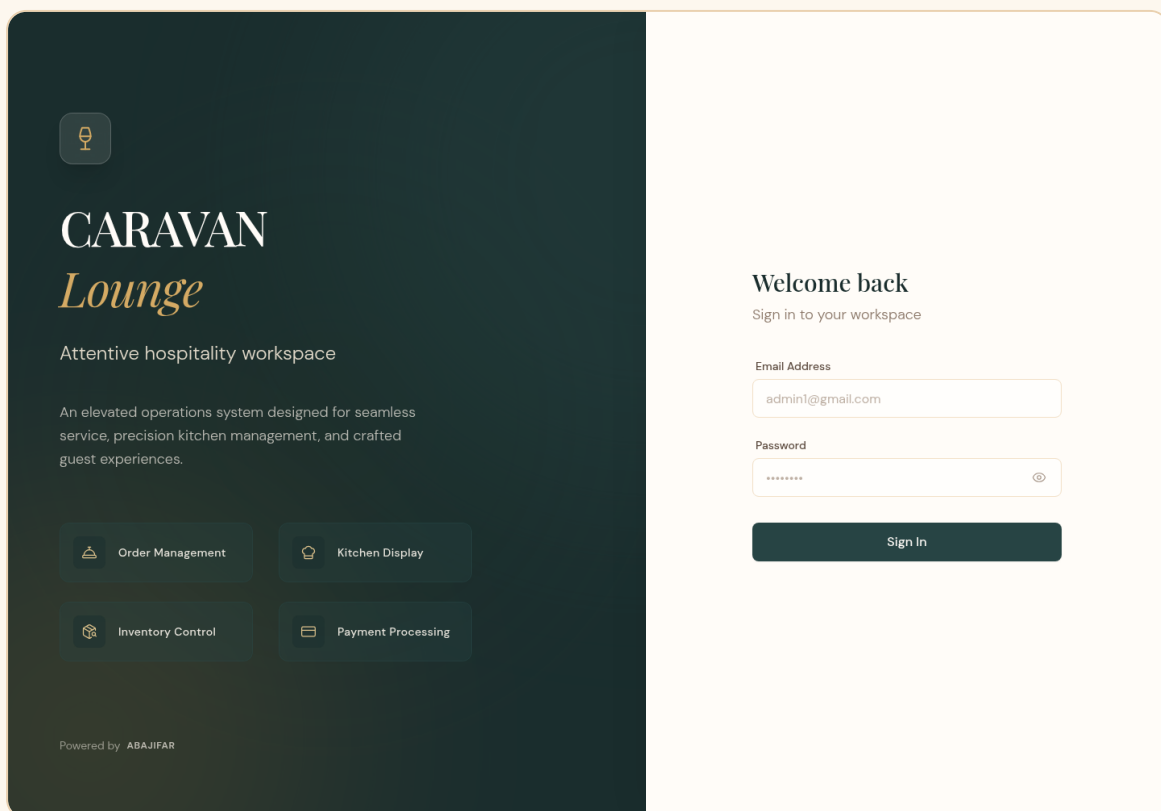
Detailed routing, environment, API, database, testing, and readiness rules are maintained in documents 07-12.

User Manual

System: Jima — CARAVAN Lounge Restaurant Management System
Version: 1.1 · **Date:** August 20, 2026

1. Signing In

Open the app in your browser. Enter the e-mail and password given to you by your manager, then press **Sign In**. What you see after logging in depends on your role — waiters, coordinators and cashiers land directly on the Orders page; managers and owners land on the dashboard.

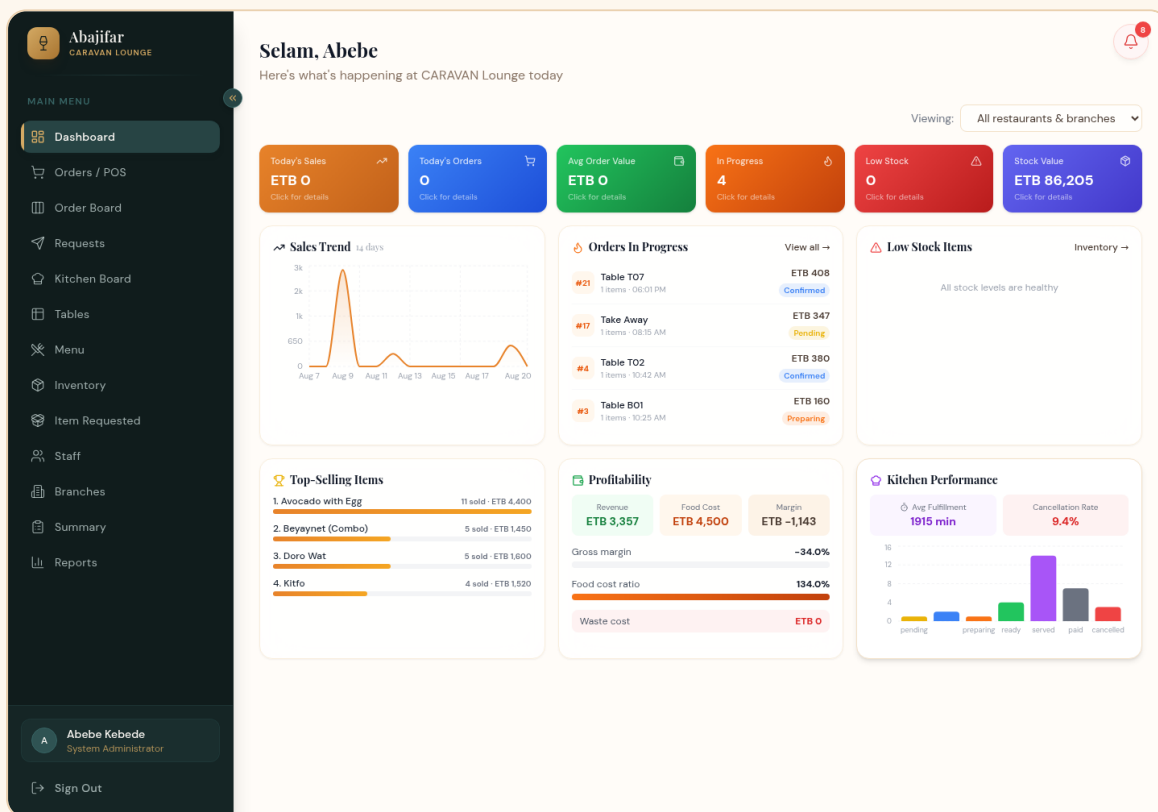


The screenshot displays the CARAVAN Lounge sign-in interface. On the left, a dark-themed sidebar contains the CARAVAN Lounge logo, a tagline 'Attentive hospitality workspace', a description of the system, and four navigation buttons: Order Management, Kitchen Display, Inventory Control, and Payment Processing. At the bottom of the sidebar, it says 'Powered by ABAJIFAR'. On the right, a light-themed login form is shown. It includes a 'Welcome back' heading, a subtitle 'Sign in to your workspace', and two input fields: 'Email Address' (containing 'admin1@gmail.com') and 'Password' (containing masked characters). A 'Sign In' button is positioned below the password field.

The sign-in page

2. Manager & Owner Dashboard

Everything important is on one page. The six colored cards at the top show today's sales, orders, average order value, orders in progress, low-stock items, and total stock value. **Click any card** to open a popup with the details behind the number. Below them: the 14-day sales trend, live orders in progress, low stock, top-selling items, profitability, and kitchen performance. The page refreshes itself every 10 seconds.



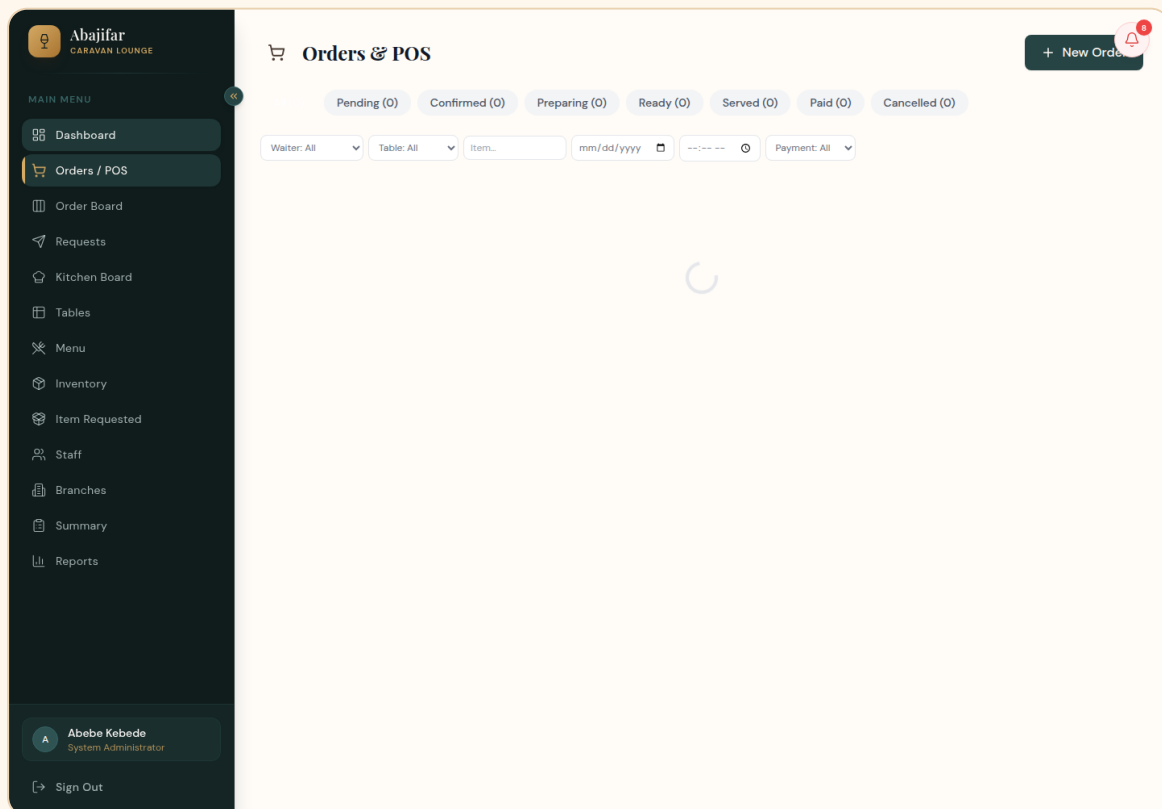
Manager / owner dashboard — click a colored card for details

Profitability compares revenue (all payments) with estimated food cost (stock deductions and waste × unit cost). **Kitchen performance** shows the average minutes from order placed to ready/served, and the cancellation rate.

3. Orders & POS

The Orders page lists every order with table, chef, item count, time, total and status. Use the tabs and filters (status, waiter, table, item,

date, time, payment) to narrow the list. Click a row to see the full order detail — it stays up to date automatically while open.



Orders list with filters; click a row for details

Taking an order (waiter)

1. Press **New Order** to open the POS.
2. Choose a table (or Take Away), tap menu items to add them, adjust quantities and notes.
3. Submit — the order appears as **pending** for the coordinator.
4. To add more items later, use **+ Add Items** on the order row (possible until it is paid).

Using the POS on a phone

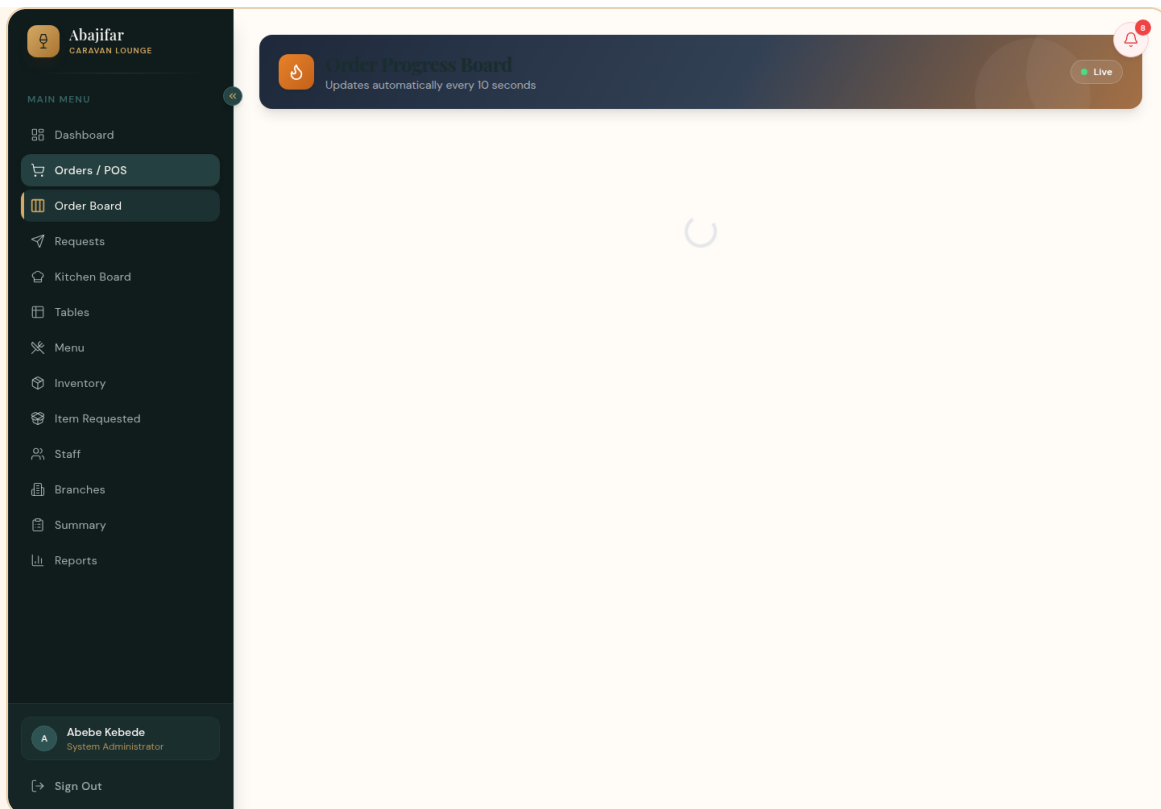
On a phone, the menu appears above the cart. Swipe the category row sideways to see more categories, scroll the menu to browse dishes, then scroll the cart section to reach notes, totals, payment details, and **Place Order**. The table, customer phone number, and waiter fields are shown one below another so they stay easy to tap. Use the ✕ at the top of the POS to close it without creating an order.

Order life cycle

Status	Meaning	Who moves it on
pending	Just placed	Coordinator confirms & assigns a chef
confirmed	Accepted, waiting for kitchen	Chef starts preparing
preparing	Being cooked	Chef marks ready
ready	Waiting to be served	Waiter serves
served	On the table	Cashier takes payment
paid	Completed	—

4. Order Progress Board

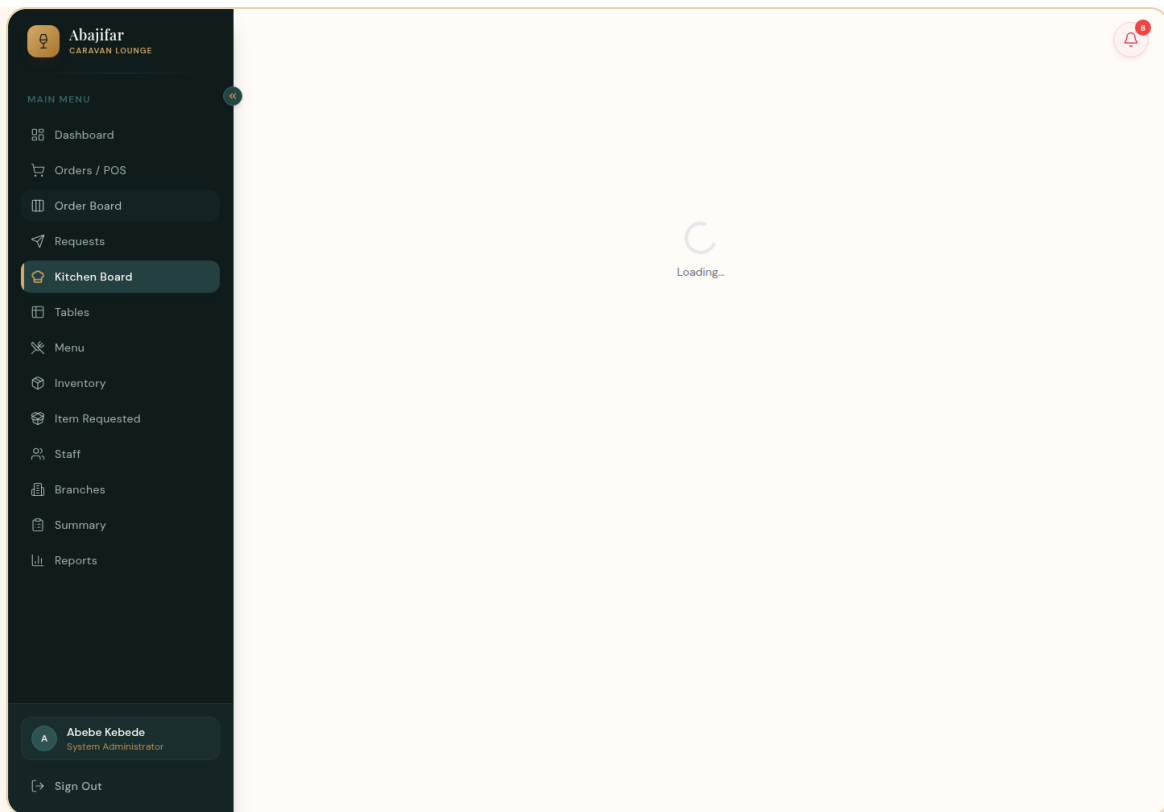
A live, color-coded view of where every active order stands: **Requested** (violet), **Received & Confirmed** (blue), **In Progress** (orange), and **Completed** (green — orders that are ready). Orders waiting more than 20 minutes get a red warning. When a column is busy it automatically switches to a compact one-line view.



The live Order Progress Board

5. Kitchen Board (chef)

Chefs see their queue of confirmed orders, grouped in tabs per chef. Use the buttons on each ticket to accept, start preparing, and mark ready. The board refreshes every 10 seconds.



Kitchen board with chef tabs

6. Tables

Shows every table with its number, seats, section, and live status. Managers can add or edit tables; tables free up automatically when an order is paid.

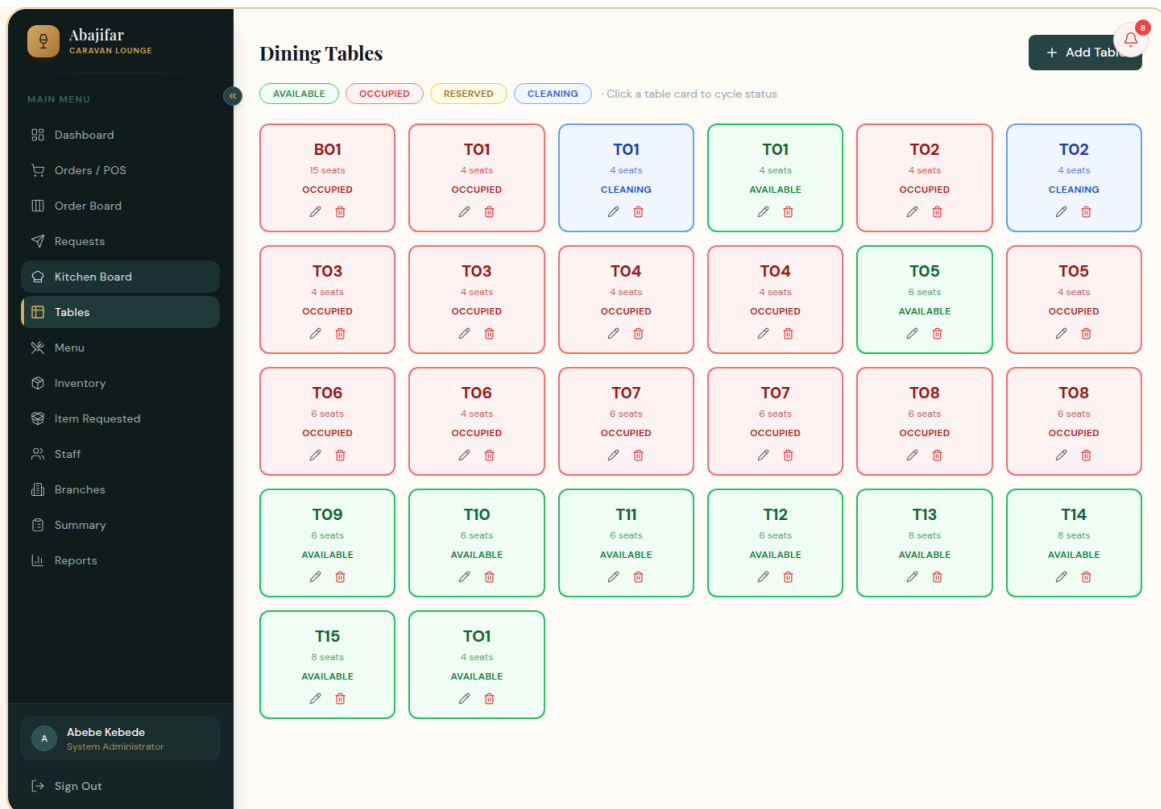
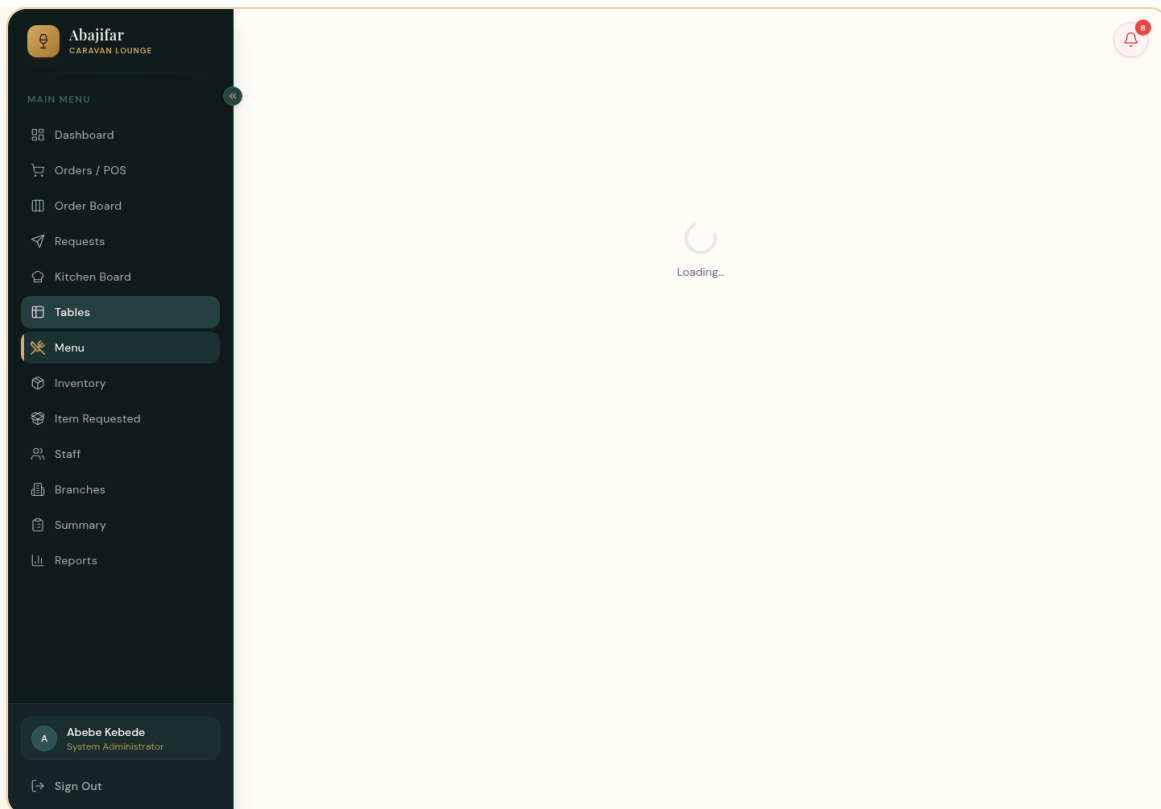


Table status overview

7. Menu (manager)

Maintain categories and dishes: name, price, description, preparation time, and availability. Items switched off here disappear from the waiters' POS immediately.



Menu management

8. Inventory (storekeeper / manager)

Track every stock item with unit, current level, minimum level, cost and expiry. Tabs cover suppliers, purchase orders, adjustments and alerts — items below minimum show as Warning, and at half the minimum (or zero) as Critical.

Abajifar
CARAVAN LOUNGE

MAIN MENU

Dashboard

Orders / POS

Order Board

Requests

Kitchen Board

Tables

Menu

Inventory

Item Requested

Staff

Branches

Summary

Reports

Abebe Kebede
System Administrator

Sign Out

Inventory Management

Items

Purchase Orders

Stock Movements

Suppliers

Alerts

Category: All categories

ITEM	CATEGORY	UNIT	STOCK	MIN STOCK	UNIT COST	TOTAL PRICE	EXPIRY	ACTIONS
Avocado	Fruits	kg	25.00	5.00	ETB 85	ETB 2,125	—	
Beef	Meat	kg	30.00	15.00	ETB 320	ETB 9,600	—	
Berberbe Spice	Spices	kg	15.00	5.00	ETB 120	ETB 1,800	—	
Chicken (Whole)	Meat	kg	30.00	10.00	ETB 180	ETB 5,400	—	
Chickpea Flour	Legumes	kg	12.00	5.00	ETB 80	ETB 960	—	
Cleaning	Cleaning	pcs	43.00	6.00	ETB 50	ETB 2,150	9/1/2026	
Ethiopian Coffee Beans	Beverages	kg	10.00	3.00	ETB 350	ETB 3,500	—	
Garlic	Vegetables	kg	5.00	2.00	ETB 120	ETB 600	—	
Honey (Tej)	Beverages	liters	20.00	5.00	ETB 280	ETB 5,600	—	
Injera	Bread	pieces	200.00	50.00	ETB 5	ETB 1,000	—	
Lamb	Meat	kg	20.00	8.00	ETB 380	ETB 7,600	—	
Mango	Fruit	KG	106.00	5.00	ETB 100	ETB 10,600	—	

+ Add Item

Inventory with stock levels and alerts

9. Item Requests

Staff request stock from the store; approvers see each request with helpful badges (low stock, expires soon, expired) and approve, reject or issue. The list updates automatically.

Abajifar

CARAVAN LOUNGE

MAIN MENU

Dashboard

Orders / POS

Order Board

Requests

Kitchen Board

Tables

Menu

Inventory

Item Requested

Staff

Branches

Summary

Reports

A

Abebe Kebede

System Administrator

Sign Out

Item Requests

Review and manage incoming item requests.

Pending Approval2

Approved5

Rejected

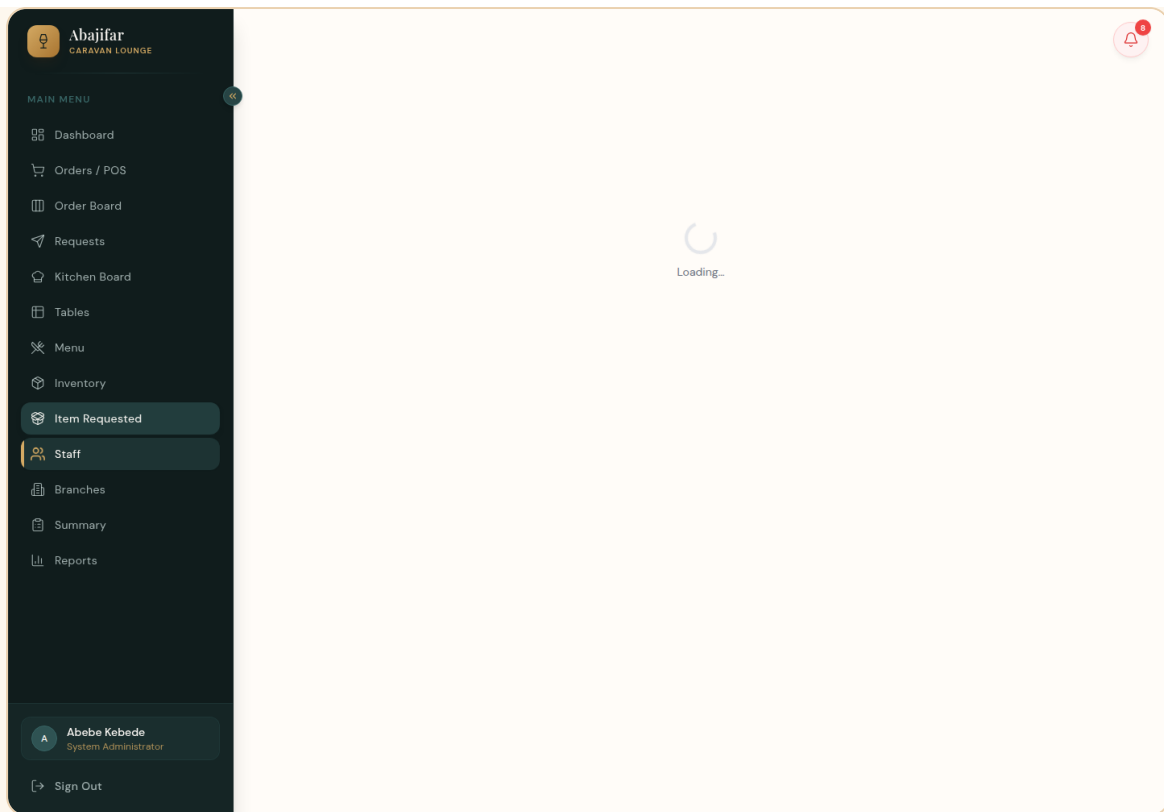
REQUESTER	ITEM DETAILS	QTY	IN STOCK	TOTAL EST.	DATE & TIME	ACTIONS
CP Cashier Piassa Branch Finance	Wheat Flour REQ-019 - adfa	10 KG	40 KG	ETB 800	8/19/2026 05:29 AM	ADJUST
WP Waiter Piassa Branch Service	Mango REQ-011 - seen	10 KG	106 KG	ETB 1,000	8/19/2026 08:33 AM	ADJUST

Showing 2 pending requests

Item request approval list

10. Staff (manager)

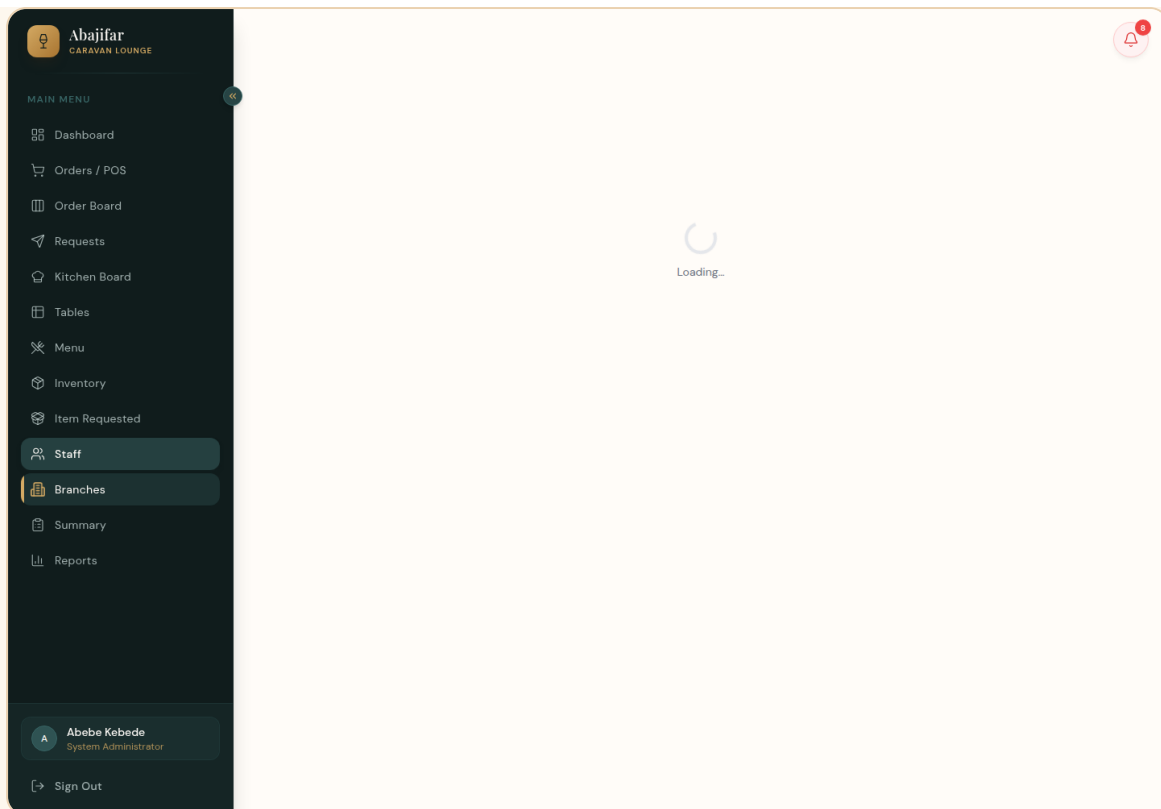
Create and manage employee accounts: name, e-mail, phone, role, branch, and active status. The header shows how many people match the current filter.



Staff management

11. Branches and Restaurants

Manage the company hierarchy: restaurants and their branches with contact details.

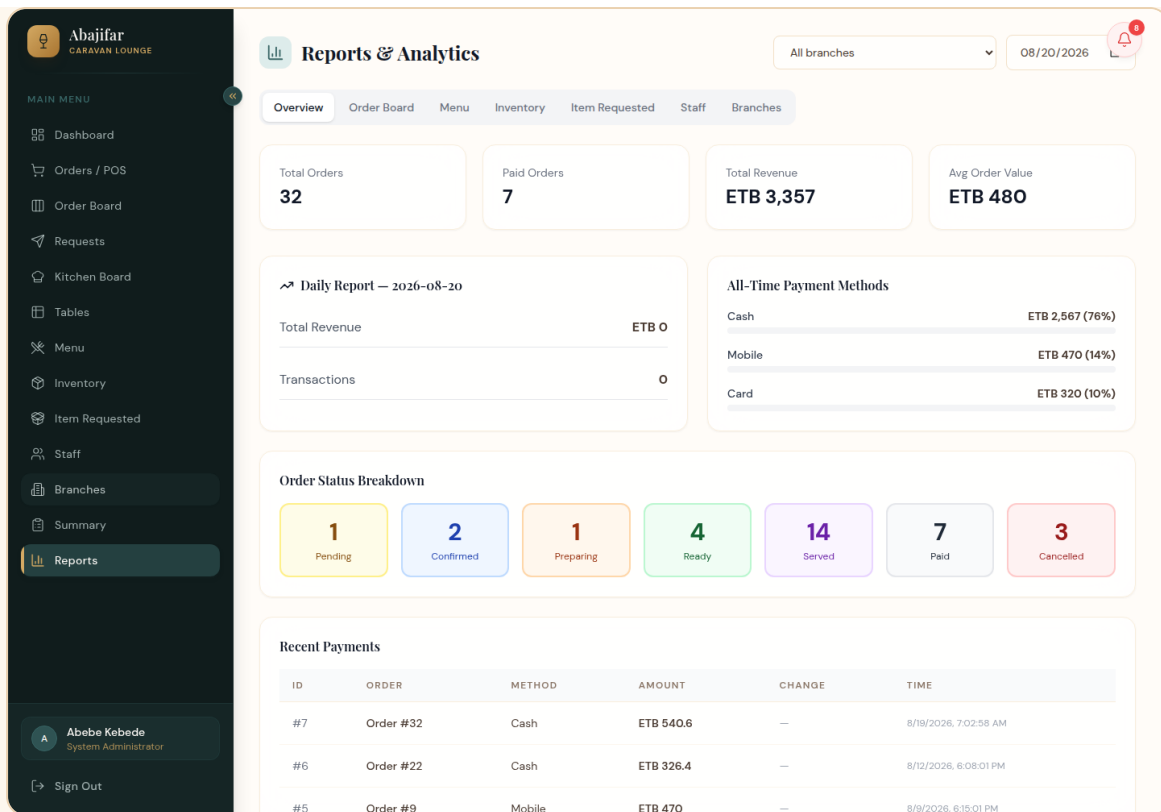


Branches and restaurants

12. Reports

Managers/owners/admins: the Overview tab shows daily revenue, payment methods and order status; the other tabs (Order Board, Menu, Inventory, Item Requested, Staff, Branches) each offer filters and one-click **Export Excel** / **Export PDF**.

Cashiers see a simplified version with just two tabs: **Sales** (filter by payment type and date) and **Items Purchased** (filter by category and date), both exportable.



Reports with filters and Excel / PDF export

13. Tips & Troubleshooting

Question	Answer
Numbers look old?	Every live page refreshes itself every 10 seconds — no refresh button needed.
Can't see a page?	Your role decides which menu entries you get. Ask a manager if you need more access.
Wrong item on an order?	While the order is pending/confirmed/preparing it can be cancelled (whole order or items) by the waiter or coordinator.
Can't see the Place Order button on my phone?	Scroll within the cart/check-out section. The menu and cart have their own scrolling areas so payment details and the final button remain reachable.
Export doesn't download?	Allow downloads/pop-ups for this site in your browser.

Forgot password?

An admin can set a new password from the Staff page.

14. Account and Support Safety

- Never share passwords in messages, screenshots, or documentation. Ask an administrator to reset access through the Staff workflow.
- If you can open a page but an action is denied, the backend role and branch rules are authoritative; contact a manager rather than retrying with another user's account.
- If a page shows old data for longer than its normal refresh period, sign out and sign in again. Report persistent network or authorization errors to the application administrator.
- The browser communicates through the application's own `/backend/api` path. Staff should never be asked for database credentials or server secrets.

Documentation update: this user manual describes operational features. Production-security and release status are tracked separately in documents 07-12 and do not change the steps above.

Project Handoff & Deployment Requirements

System: Jima — CARAVAN Lounge Restaurant Management System

Document: Handoff v1.2 · **Updated:** August 20, 2026

1. Repository Layout

```
workspace/ (pnpm monorepo)
├─ artifacts/
│   ├─ habesha/           # Registered Jima web artifact / routing shim
│   ├─ nextjs-app/        # Real frontend – Next.js 15, React 19
│   │   ├─ src/app/        # Pages: login, dashboard/* (orders, kitchen, ...
│   │   │   ├─ src/components # Sidebar, manager-dashboard, shared UI
│   │   │   ├─ src/lib/api.ts # All REST calls (Axios + JWT header)
│   │   │   ├─ src/store/    # Zustand auth store
│   │   │   └─ public/       # Static assets (login-bg.jpg, ...)
│   └─ nestjs-server/     # Authoritative restaurant API – NestJS 10
│       └─ src/<module>/    # auth, users, orders, kitchen, payments,
│                           # menus, tables, inventory, branches,
│                           # restaurants, notifications, seed
│   └─ api-server/        # Separate minimal artifact; not the restaurant API
├─ docs/                 # This documentation set + screenshots
└─ package.json          # Workspace root
```

2. How to Run (Development)

```
# prerequisites: Node.js 18+, pnpm 8+, PostgreSQL 14+
pnpm install

# backend (port 3001, API prefix /api, Swagger at /api/docs)
DATABASE_URL=postgres://user:pass@host:5432/db \
JWT_SECRET=change-me \
pnpm --filter @workspace/nestjs-server run dev
```

```
# frontend (port 3000; proxies /backend/api/* to the backend)
BACKEND_URL=http://localhost:3001 \
pnpm --filter @workspace/nextjs-app run dev
```

Development seed operations require administrative authentication. Do not publish or reuse development credentials. Production provisioning must be controlled, auditable, and separate from normal application startup.

3. Environment Variables

Variable	Used by	Purpose
DATABASE_URL	Backend	PostgreSQL connection string. Append <code>?sslmode=require</code> for managed databases (SSL is honored automatically).
JWT_SECRET	Backend	Signing key for access tokens. Must be supplied as a secret; the current fallback is a production blocker.
PORT	Both	Listen port (backend defaults to 3001).
BACKEND_URL	Frontend	Where the Next.js rewrite proxies API calls (e.g. <code>https://api.example.com</code>).
CORS_ORIGIN	Backend	Allowed production frontend origin after wildcard CORS is removed.

4. Deployment Requirements

Requirement	Detail
Node runtime	Node.js 18 or newer, pnpm available at build time.
Database	PostgreSQL 14+ (managed service recommended: Neon, Supabase, RDS, Cloud SQL). ~100 MB is plenty to start.
Processes	

	Two services: frontend (Next.js server or Vercel) and backend (NestJS, long-running Node process).
Networking	Frontend must reach the backend over HTTPS; browser only ever talks to the frontend origin (same-origin proxy).
TLS	Terminate HTTPS at the platform/load balancer.
Memory	512 MB per service is sufficient for small deployments.

5. Production Checklist

✓	Item
☐	Require a strong <code>JWT_SECRET</code> and remove the hard-coded fallback.
☐	Stop startup password resets/account reactivation; keep seed and account provisioning controlled and production-gated.
☐	Turn off TypeORM <code>synchronize</code> and adopt migrations (<code>typeorm migration:generate</code>) — auto-sync on a live database can drop/alter columns unexpectedly.
☐	Enable database backups (daily snapshot is enough initially).
☐	Restrict CORS on the backend to the frontend's production origin.
☐	Restrict or disable Swagger and seed operations in production.
☐	Resolve high security-scan findings or record a reviewed risk acceptance.
☐	Run automated role, branch, order, payment, inventory, report, notification, and export checks.
☐	Verify the Orders/POS flow at 320 px and 390 px mobile viewports: menu/cart stack cleanly, checkout controls do not overlap, and the final action remains reachable.
☐	Set up log retention / error alerting on both services.
☐	Create real staff accounts with strong passwords; deactivate demo users.

Data safety: the system currently relies on `synchronize:true`. Before the first production deploy, generate the schema once, then disable synchronize. Never point a synchronized dev backend at the production database.

6. Operational Notes for the Next Team

- **Verification habit:** run `npx tsc --noEmit` inside `artifacts/nextjs-app` after frontend changes. Avoid running `next build` while the dev server is running.
- **Polling:** all live pages poll every 10 s; there is no WebSocket layer to maintain.
- **Exports:** Excel/PDF exports happen in the browser (xlsx, jsPDF) — no server-side rendering service is needed.
- **Role logic:** sidebar visibility lives in `src/components/sidebar.tsx`; per-page role behavior is in each page component; backend guards are authoritative.
- **Mobile POS:** the order form changes from two columns to a vertical menu-and-cart layout on phones. Preserve the separate menu/cart scrolling areas and single-column cart fields when changing the POS.
- **Profitability math:** food cost is estimated from stock deduction/waste movements $\times$ unit cost; accuracy depends on the store team recording usage.
- **API reference:** Swagger UI is served by the backend at `/api/docs`.

7. Support Artifacts Included

File	Content
docs/01-FRS...html	Functional requirements by subsystem and role
docs/02-SRS...html	Full software requirements incl. non-functional

docs/03-SDD...html	Architecture, ER, and sequence diagrams; design decisions
docs/04-User-Manual.html	Step-by-step guide with screenshots
docs/06-Deploy-on-Vercel.html	Vercel-specific deployment walkthrough
docs/07-Architecture-and- Routing.md	Canonical artifact, routing, and service ownership
docs/08-Environment-and- Secrets.md	Variable ownership and secret-handling rules
docs/09-API-and- Authorization-Boundary.md	API paths, authentication, authorization, and error contracts
docs/10-Database-Safety- Runbook.md	Migration, backup, restore, seed, and rollback controls
docs/11-Test-and-Release- Runbook.md	Required build, test, security, smoke, and rollback gates
docs/12-Production-Readiness- Status.md	Current READY / NEEDS ATTENTION / BLOCKED evidence matrix

Deploying on Vercel

System: Jima — CARAVAN Lounge Restaurant Management System
Document: Alternative Deployment Guide v1.1 · **Updated:** August 20, 2026

1. What goes where

Status: BLOCKED. This is an alternative deployment reference, not approval to deploy. Complete all mandatory controls in documents 10-12 before using these instructions. Current blockers include schema synchronization, authentication defaults, startup account mutation, wildcard CORS, and incomplete regression evidence.

Vercel is built for frontend frameworks. This system has **two services**, so the recommended split is:

Piece	Where it runs
Next.js frontend (artifacts/nextjs-app)	Vercel — first-class Next.js support.
NestJS backend (artifacts/nestjs-server)	A long-running Node host: Railway, Render, Fly.io , or a small VM. (Vercel serverless functions are not a good fit for a full NestJS app with TypeORM connections.)
PostgreSQL	Managed database: Neon , Supabase, Railway Postgres, or RDS.

2. Step 1 — Database

1. Create a PostgreSQL database (e.g. Neon free tier).

2. Copy the connection string, e.g.

```
postgres://user:pass@host/db?sslmode=require — keep  
sslmode=require ; the backend enables SSL automatically when it  
sees it.
```

3. Step 2 — Backend (Railway example)

1. Push the repository to GitHub.
2. In Railway: **New Project → Deploy from GitHub**, root directory `artifacts/nestjs-server`.
3. Build & start commands:

```
Build:  pnpm install && pnpm run build  
Start:  node dist/main.js
```

4. Environment variables:

```
DATABASE_URL = postgres://...?sslmode=require  
JWT_SECRET   = <long random string>  
PORT         = 3001    (or leave to platform default)
```

5. Deploy only after the release gate is approved. Verify the protected health and API behavior without exposing Swagger publicly.
6. Provision initial production accounts through the approved controlled process. Do not call the development seed endpoint against production.

4. Step 3 — Frontend on Vercel

1. In Vercel: **Add New → Project**, import the GitHub repository.
2. **Root Directory:** set to `artifacts/nextjs-app` (Vercel detects Next.js automatically).
3. **Install command:** `pnpm install` (Vercel supports pnpm monorepos; enable "Include files outside root directory" if prompted).
4. **Environment variables:**

```
BACKEND_URL = https://jima-api.up.railway.app
```

The frontend's `next.config.js` rewrite forwards `/backend/api/*` to this URL. Backend CORS must still be restricted as defense in depth and for any direct operational access.

5. Click **Deploy**. Your app is live at `https://<project>.vercel.app`.

5. Step 4 — Verify

Check	Expected
Open the Vercel URL	Login page with photo background loads.
Log in as admin	Dashboard renders with data (KPI cards populated).
Browser dev tools → Network	API calls go to <code>/backend/api/...</code> on the same domain and return 200.
Create a test order	Appears on the Order Progress Board within 10 s.

6. Custom domain (optional)

1. Vercel → Project → Settings → Domains → add `app.yourdomain.com` and follow the DNS instructions.
2. HTTPS certificates are automatic.

7. Updates

Every push to the connected GitHub branch redeploys the frontend automatically on Vercel (and the backend on Railway). Preview deployments are created for pull requests.

Before going live: complete the production checklist and evidence matrix in documents 05 and 12. Do not deploy while any mandatory item is marked BLOCKED.

8. Troubleshooting

Symptom	Likely cause / fix
Login fails with network error	<code>BACKEND_URL</code> missing or wrong on Vercel; redeploy after fixing (rewrites are baked at build time).
500s from API	Backend can't reach the database — check <code>DATABASE_URL</code> and that <code>sslmode=require</code> is present for managed Postgres.
Build fails on Vercel	Ensure Root Directory is <code>artifacts/nextjs-app</code> and <code>pnpm</code> is selected; clear build cache and retry.
Works locally, 404 on API in production	Calls must go through <code>/backend/api/*</code> ; hardcoded <code>localhost:3001</code> URLs won't work in production.